

Enterprise Claude Skills - Requirement Gap Analysis

Scope

This compares the handbook requirements against what is currently implemented in the `Cloude-Skills-agents` repo.

Current Standing (high level)

- **Foundation is built:** control-plane style backend + admin UI + PostgreSQL schema + demo seed data.
- **Core catalog/packaging entities exist:** skills, packages, suites, overlays, entitlements, agents, runs, audit, usage charges.
- **Governance primitives exist:** trust/review/quarantine fields, activation checks, approval/governance reporting.
- **Not enterprise-complete yet:** real auth/RBAC, workflow-grade approvals engine, workers, integrations, search/RAG, scale hardening.

What is Implemented and Working

1) Data model and architecture (foundational)

- Multi-module Postgres schema via migrations:
 - 0002_skill_registry.sql
 - 0003_department_suites_overlays.sql
 - 0004_commercial.sql
 - 0005_routing.sql
 - 0005_skill_runs_audit.sql
 - 0006_security.sql
 - 0007_demo_seed.sql
- Backend route modules for:
 - registry, suites/overlays, entitlements/commercial, agents, tasks, routing, runs, security, reports, health.
- Admin frontend pages for key entities and reporting.

2) API data pulling

- REST read endpoints are present and consumed by the frontend list/report pages.
- Frontend API client is wired through `REACT_APP_API_BASE`.
- Health endpoint and API routes are active when backend is running.

3) Commercial/control-plane base

- Entitlements, subscriptions/plans, credit pools, usage charging ledger, and billing-style reports exist.

- Demo catalog + demo customer/workspace + sample run/audit data are seeded.

4) Governance base

- Skill security metadata, scan history table, review/quarantine/pinning endpoints.
- Activation guard checks for unsafe/unreviewed/unpinned skills.
- Audit log and governance metrics endpoints.

Partial / Basic-Only (implemented but not enterprise-grade yet)

A) Auth and access control

- Current UI guard is token in localStorage (basic).
- No real identity provider integration.
- No server-enforced role matrix/RBAC across API actions.

B) Approval system

- Approval concepts exist in schema/reporting.
- Dedicated approval queue workflow, SLA/escalation workers, and full reviewer lifecycle are not complete.

C) Orchestration/runtime

- Routing and run tracking exist.
- No production worker pipeline / queue orchestration loop implemented end-to-end.

D) Reporting UX

- Reporting endpoints exist.
- UI is functional but table/basic-view style; charts/CSV/export experience is not fully built.

Major Handbook Requirements Not Yet Implemented

1. **Enterprise identity and security**
 - SSO/SAML/OIDC, strict RBAC, policy-bound permissions, API keys/rate limits.
2. **Knowledge + AI search layer**
 - Brand-voice knowledge base, hybrid lexical+semantic retrieval, vector index management.
3. **Integration layer**
 - Asana/monday.com/GitHub/Slack connectors, OAuth lifecycle, web-hook reliability framework.

4. **True approval/policy engine**
 - Risk-tiered action enforcement with complete state machine, escalations, and admin policy tooling.
5. **Worker/scaling hardening**
 - Redis/BullMQ operationalized in runtime, background jobs, retries, dead-letter handling, observability.
6. **Admin vs end-user split maturity**
 - Admin shell exists; end-user workflow collaboration surface is still minimal.
7. **Enterprise observability/compliance**
 - OpenTelemetry-level tracing/alerts, immutable audit controls, evidence exports.

Where We Stand vs Your V1 Core Loop

From the handbook “v1 should prove 10 things”:

1. Tenant/workspace creation -> **Partial** (workspace/commercial entities exist; auth/org maturity pending)
2. Activate department suite -> **Partial** (models + routes exist; policy UX limited)
3. Activate industry overlay -> **Partial** (models + routes exist; policy UX limited)
4. Use approved skills/agents -> **Partial** (registry/agents/security base exists)
5. Route tasks to agents -> **Partial** (task/routing/run modules exist; runtime depth limited)
6. Runs logged + metered -> **Implemented**
7. High-risk approvals required -> **Partial** (foundations exist, full engine pending)
8. Skill governance lifecycle -> **Partial-to-Implemented** (review/quarantine/pinning exists, full lifecycle ops pending)
9. Admin can see audit + usage -> **Implemented (basic UI)**
10. Real external integrations -> **Not implemented yet**

Recommended Next Execution Plan

Phase 1 (immediate hardening)

1. Real auth + API RBAC.
2. Approvals API + queue page backed by real workflow states.
3. Run worker service with Redis/BullMQ and retry/escalation logic.

Phase 2 (enterprise capability)

4. Integration framework: GitHub + Slack first, then Asana/monday.
5. API key + rate limiting + connector credential vaulting.
6. Reporting upgrades (charts, export, governance drill-down).

Phase 3 (AI operating-system depth)

7. Knowledge ingestion + hybrid search + brand voice retrieval.
8. Policy bundles by department/industry/risk tier.
9. Strong observability + compliance evidence exports.

Bottom Line

- The repo is a **strong control-plane foundation** and **demoable v1 base**.
- It is **not yet a full enterprise AI operating system** from the handbook.
- Main gap is not CRUD; main gap is **enterprise governance depth + integrations + runtime hardening**.